

IND320 - Project Work Part 1

Name: Yamama Alhalabi

GitHub repository: <https://github.com/yamamaalhalabi/ind320-yamamaalhalabi>

Streamlit app: <https://ind320-yamamaalhalabi.streamlit.app/>

Screencast: https://github.com/yamamaalhalabi/ind320-yamamaalhalabi/blob/main/IND320_screencast.mp4

```
In [1]: import pandas as pd

# Read weekly reservoir data and keep only the national ("NO") level,
# since the raw file mixes several regional breakdowns (NO, EL, VASS)
df = pd.read_csv("reservoirs.csv", parse_dates=["dato_Id"])
df = df[df["omrType"] == "NO"].copy()
df = df.sort_values("dato_Id")

# Rename columns to English, understandable names
df = df.rename(columns={
    "dato_Id": "date",
    "fyllingsgrad": "fill_ratio",
    "kapasitet_TWh": "capacity_twh",
    "fylling_TWh": "fill_twh",
    "fyllingsgrad_forrige_uke": "fill_ratio_prev_week",
    "endring_fyllingsgrad": "fill_ratio_change",
})

df = df.set_index("date")
df = df[["fill_ratio", "capacity_twh", "fill_twh",
        "fill_ratio_prev_week", "fill_ratio_change"]]

df.head()
```

```
Out[1]:
```

	fill_ratio	capacity_twh	fill_twh	fill_ratio_prev_week	fill_ratio_change
date					
1995-01-08	0.608274	87.43758	53.185986	0.752703	-0.144430
1995-01-15	0.582372	87.43758	50.921207	0.608274	-0.025902
1995-01-22	0.562412	87.43758	49.175980	0.582390	-0.019977
1995-01-29	0.534508	87.43758	46.736122	0.562412	-0.027904
1995-02-05	0.506486	87.43758	44.285900	0.534489	-0.028003

Printing the contents

```
In [2]: print(f"Antall rader: {df.shape[0]}, antall kolonner: {df.shape[1]}")
print(f"Periode: {df.index.min().date()} til {df.index.max().date()}")
df.describe()
```

Antall rader: 1653, antall kolonner: 5

Periode: 1995-01-08 til 2026-09-06

```
Out[2]:
```

	fill_ratio	capacity_twh	fill_twh	fill_ratio_prev_week	fill_ratio_change
count	1653.000000	1653.000000	1653.000000	1653.000000	1653.000000
mean	0.622086	87.43758	54.393716	0.622156	-0.000070
std	0.195967	0.000000	17.134922	0.195992	0.027704
min	0.180250	87.43758	15.760602	0.180260	-0.144430
25%	0.460607	87.43758	40.274364	0.460607	-0.022063
50%	0.645902	87.43758	56.476093	0.647445	-0.005650
75%	0.789274	87.43758	69.012170	0.789271	0.016337
max	0.966916	87.43758	84.544820	0.966892	0.125069

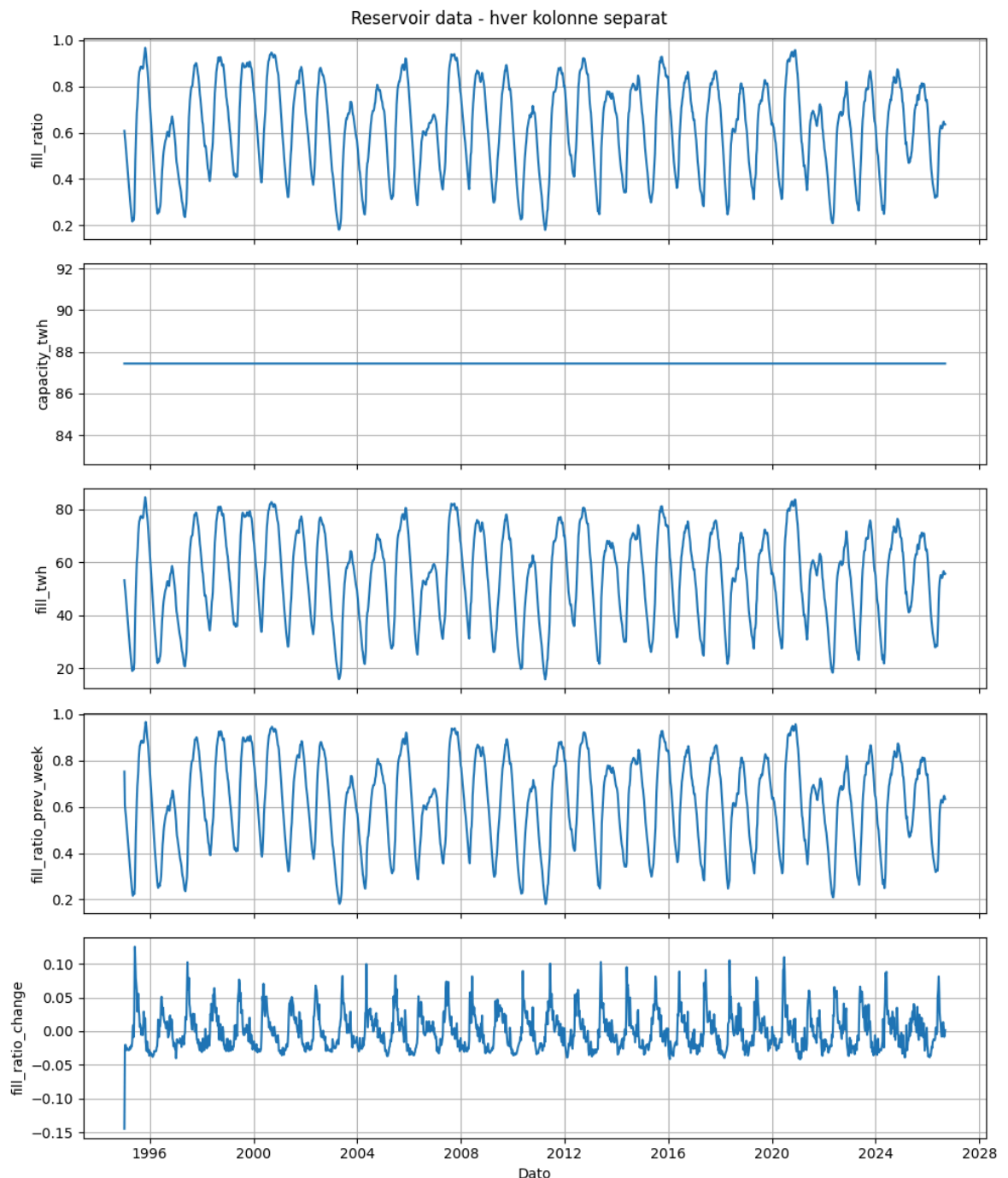
Plotting each column separately

```
In [3]: import matplotlib.pyplot as plt

fig, axes = plt.subplots(len(df.columns), 1, figsize=(10, 12), sharex=True)

for ax, col in zip(axes, df.columns):
    ax.plot(df.index, df[col])
    ax.set_ylabel(col)
    ax.grid(True)

axes[-1].set_xlabel("Dato")
fig.suptitle("Reservoir data - hver kolonne separat")
plt.tight_layout()
plt.show()
```



Plotting all columns together

Since the columns have very different scales (e.g. `capacity_twh` is around 87, while `fill_ratio_change` is close to 0), each column is normalized to the same 0-1 scale before being shown in the same graph. This makes it possible to compare the patterns over time, even though the actual values differ greatly.

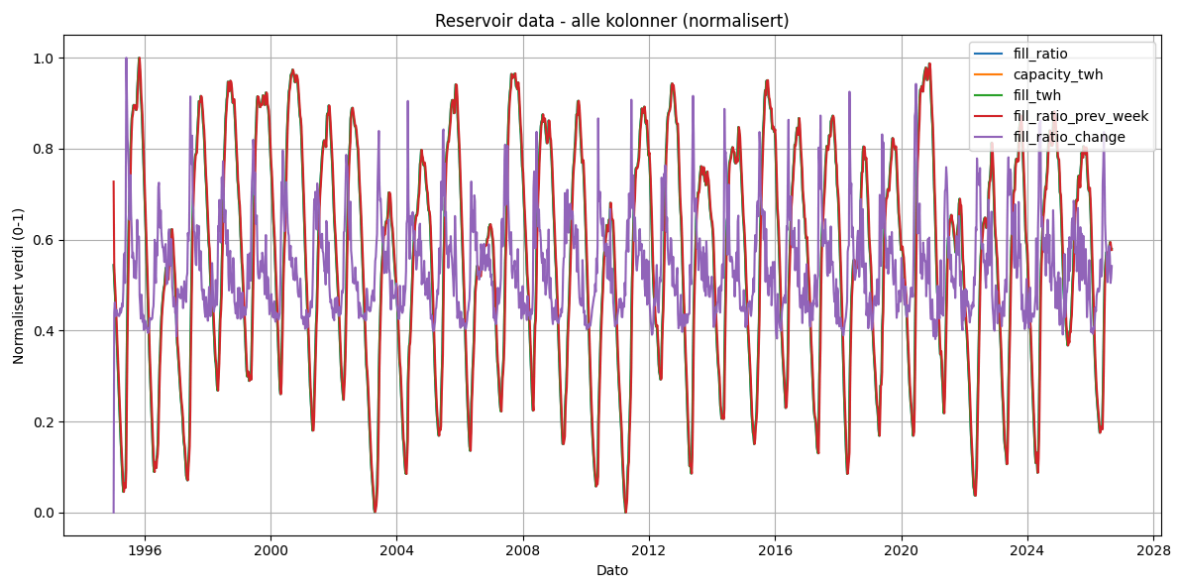
```
In [4]: # Normalize each column to a 0-1 scale so they can be compared
# despite having very different units and value ranges
df_normalized = (df - df.min()) / (df.max() - df.min())

fig, ax = plt.subplots(figsize=(12, 6))
for col in df_normalized.columns:
    ax.plot(df_normalized.index, df_normalized[col], label=col)
```

```

ax.set_xlabel("Dato")
ax.set_ylabel("Normalisert verdi (0-1)")
ax.set_title("Reservoir data - alle kolonner (normalisert)")
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()

```



Log

In this project, I set up the basic structure for a Streamlit-based project in IND320. The work started by creating a public GitHub account and repository (ind320-yamamaalhalabi), followed by a Streamlit Cloud account connected to the same GitHub account. I built a minimal Streamlit app with a simple front-page text, pushed it to GitHub, and got it running at ind320-yamamaalhalabi.streamlit.app. I then expanded the app to four pages using Streamlit's automatic sidebar navigation: a front page, plus the pages Table, Plot, and About, each with its own .py file in a pages folder.

Along the way, I encountered an encoding error when PowerShell saved a file containing Norwegian characters (å) in the wrong character set, causing a utf-8 error in the running app. I resolved this by specifying UTF-8 encoding explicitly, and then switched to editing files in VS Code instead of directly in PowerShell, since VS Code saves in UTF-8 by default.

The data used in this project is reservoirs.csv, obtained from the course's GitHub repository (khliland/IND320). The file contains weekly figures for the fill level of Norwegian water reservoirs, broken down into several regional levels (national, electricity price areas, and watercourse regions). I filtered the data to the national level only, to get one continuous time series per column, and renamed the columns from Norwegian to understandable English names (e.g. fyllingsgrad to fill_ratio).

In Jupyter Notebook, I read and cleaned the data using Pandas, printed basic statistics with describe(), and created two types of plots: one showing each column separately in its own subplot, and one showing all columns together in a single graph. Since the

columns have very different scales (e.g. capacity in TWh versus a change rate close to zero), I normalized the values to a common 0-1 range before plotting them together, so the patterns could be compared visually.

On the Streamlit side, I implemented a table on page 2 that shows each column as its own row, with a line-chart sparkline (LineChartColumn) showing the development over the first month of data. On page 3, I built an interactive plot with a dropdown menu (selectbox) to select either a single column or all columns at once, as well as a select_slider to choose which time period (months) to display in the graph.

Throughout the process, I used AI tools to help with debugging, code structure, and explanations of Streamlit and Pandas functionality, as well as to locate the correct location for the course's data files. All code has been tested both locally and in the deployed Streamlit Cloud app to confirm it works consistently in both environments.

Use of AI

Claude (Anthropic) was used throughout the project to help with Python code (Pandas, Plotly, Streamlit), debugging error messages (including encoding issues in PowerShell), explaining Git/GitHub commands, and locating the correct source file (reservoirs.csv) in the course's GitHub repository.